



UMN

INSTITUTO POLITÉCNICO DA HUÍLA

DEPARTAMENTO DE INFORMÁTICA E COMPUTAÇÃO

Departamento de Informática e Computação

LABORATÓRIO DE PROGRAMAÇÃO AVANÇADA

BACKTRACING

UMA TÉCNICA DE RESOLUÇÃO DE PROBLEMAS COMPUTACIONAIS

Lubango 2026

Departamento de Informática e Computação

LABORATÓRIO DE PROGRAMAÇÃO AVANÇADA

BACKTRACING

UMA TÉCNICA DE RESOLUÇÃO DE PROBLEMAS COMPUTACIONAIS

Curso de Ciência da Computação — 3.º Ano

Trabalho Prático N.º

Integrantes:

- Dário Puíssa João — 2023100718
- Félix Firmino Bongue-2023142258

Docente:

Lubango 2026

RESUMO

A computação contemporânea enfrenta diariamente problemas cuja complexidade inviabiliza a aplicação de métodos diretos ou algoritmos determinísticos simples. Neste contexto, as técnicas de busca sistemática em espaços de estados emergem como ferramentas indispensáveis. O presente trabalho tem como objeto de estudo o backtracking, uma estratégia algorítmica baseada na construção incremental de soluções combinada com a capacidade de retroceder quando determinada escolha se revela inadequada ao objetivo pretendido.

Diferentemente da abordagem ingênua de força bruta, que enumera exaustivamente todas as combinações possíveis antes de qualquer verificação, o backtracking introduz o conceito de poda a eliminação precoce de ramos inteiros da árvore de busca que jamais poderiam conduzir a uma solução válida. Esta característica, quando bem explorada, pode reduzir drasticamente o esforço computacional necessário, transformando problemas intratáveis em desafios resolvíveis dentro de horizontes de tempo aceitáveis.

A pesquisa percorre desde os fundamentos teóricos da técnica suas origens históricas, condições de aplicabilidade e representação como árvore de busca até aspectos práticos de implementação e análise de complexidade. São examinadas as principais variações do método, incluindo backjumping, heurísticas de ordenamento e propagação de restrições, bem como suas limitações inerentes, com destaque para a suscetibilidade à explosão combinatória em problemas de grande escala.

Conclui-se que o backtracking, embora não seja uma panaceia para todos os problemas computacionais, constitui ferramenta de valor inestimável no repertório do cientista da computação, especialmente quando combinado com heurísticas adequadas ao domínio de aplicação. Sua elegância conceitual e relativa simplicidade de implementação contrastam com a profundidade dos problemas que permite resolver, justificando seu estudo sistemático em cursos de algoritmos e estruturas de dados.

Sumário

INTRODUÇÃO	6
1.1 OBJETIVOS	6
1.2 JUSTIFICATIVA	7
2 CAPÍTULO I: FUNDAMENTAÇÃO TEÓRICA.....	8
2.1 Definição Formal E Breve Histórico	9
2.2 A Árvore De Busca Como Modelo De Compreensão.....	9
2.3 Recursão E Backtracking.....	10
2.4 As Funções Constituintes Do Algoritmo.....	11
2.4.1 A função de rejeição (reject)	11
2.4.2 A função de aceitação (accept).....	11
2.4.3 A função de geração da primeira extensão (first)	11
2.5 Onde O Backtracking Se Aplica.....	12
2.5.1 Estrutura de decisão sequencial:	12
2.5.2 Testabilidade precoce:.....	12
2.5.3 Domínios pequenos	12
2.5.4 Soluções completas necessárias:	12
2.6 Onde o backtracking Não Se Aplica.....	12
2.6.1 Problemas de otimização contínua	12
2.6.2 Problemas onde a poda é ineficaz	13
2.6.3 Problemas que admitem algoritmos gulosos ótimos	13
2.6.4 Problemas muito grandes	13
2.7 Poda	13
2.8 Tipos de Poda	14
2.8.1 Poda por restrições locais	14
2.8.2 Poda por forward checking	14
2.8.3 Poda por limite (branch and bound	14
2.8.4 Poda por simetria.....	14
2.9 Complexidade.....	15
2.10 Variantes E Aprimoramentos.....	16
2.10.1 Backjumping	16
2.10.2 Conflict-Directed Backjumping (CBJ).....	16
2.10.3 Forward checking com propagação de restrições	16
3 CAPÍTULO II :ESTUDOS DE CASO	17
3.1 O Problema Das N-Rainhas.....	18

3.2	O Problema Do Suíte E A Colocação De Rainhas Modificada	Error! Bookmark not defined.
3.3	O Problema Do Caixeiro Viajante.....	19
3.4	O Problema Da Mochila (0/1)	19
4	Conclusão	20
5	Bibliografia.....	21

INTRODUÇÃO

Pensemos, na tarefa de posicionar oito rainhas em um tabuleiro de xadrez de modo que nenhuma delas ataque outra. Não se trata de um problema trivial: embora a primeira rainha possa ser colocada em qualquer uma das sessenta e quatro casas, cada posicionamento subsequente elimina diversas alternativas, e o número total de arranjos possíveis é astronômico. Como encontrar uma solução sem testar todas as combinações? Eis aí o espaço onde o backtracking floresce.

O presente trabalho propõe-se a examinar em profundidade esta técnica, investigando não apenas seus mecanismos internos, mas também o domínio de problemas para os quais se mostra particularmente adequada. A motivação para este estudo reside na constatação de que, embora amplamente ensinada em cursos introdutórios de algoritmos, a técnica é frequentemente reduzida a um punhado de exemplos canônicos (o problema das N-rainhas, o preenchimento de Sudoku, a geração de permutações), sem que seus fundamentos mais sutis — as condições de poda, a escolha das heurísticas, a relação entre recursão e retrocesso — sejam adequadamente aprofundados.

1.1 OBJETIVOS

Objetivo Geral

Analisar o backtracking enquanto técnica algorítmica de resolução de problemas computacionais, compreendendo seus fundamentos teóricos, seu campo de aplicabilidade, suas variações e suas limitações, de modo a obter uma base sólida para aplicação consciente e eficaz do método.

Objetivos Específicos

Descrever o funcionamento do algoritmo por meio da representação como árvore de busca, explicitando os papéis das funções de rejeição, aceitação e geração de extensões.

Identificar as condições necessárias para que um problema seja adequadamente resolvido por backtracking, bem como os sinais de que outras técnicas seriam mais apropriadas.

Apresentar exemplos clássicos de aplicação, demonstrando a tradução dos conceitos abstratos em implementações concretas.

1.2 JUSTIFICATIVA

Por que dedicar tempo e esforço ao estudo aprofundado do backtracking? A pergunta é legítima, especialmente quando se considera que, nas últimas décadas, surgiram técnicas mais sofisticadas de programação linear inteira, meta-heurísticas como algoritmos genéticos e busca tabu, métodos de aproximação que em muitos contextos superam o desempenho do backtracking.

A resposta reside em dois fatos fundamentais. O primeiro é que o backtracking, ao contrário de muitas abordagens modernas, oferece garantias de exaustividade quando o espaço de busca é finito e a função de rejeição não produz falsos negativos, o algoritmo encontra todas as soluções do problema, ou certifica que nenhuma existe. Esta propriedade é inestimável em contextos onde a completude é exigida em verificação formal, em planejamento crítico, em problemas onde soluções parciais ou aproximadas são insuficientes.

O segundo fato é que o backtracking constitui a base conceitual sobre a qual técnicas mais avançadas foram construídas. Compreender o backtracking em profundidade é um pré-requisito para compreender programação por restrições, satisfabilidade booleana (SAT) e algoritmos de busca em jogos (como minimax com poda alfa-beta). O estudante que domina o backtracking está equipado para avançar nestes campos com muito mais fluência do que aquele que aprendeu a técnica apenas como um conjunto de receitas.

Ademais, o problema da explosão combinatória que assola o backtracking não é uma falha do método, mas uma manifestação da complexidade intrínseca dos problemas resolvidos. Para a vasta classe dos problemas NP completos, não se conhece algoritmo de tempo polinomial, e o backtracking representa, em muitos casos, a melhor alternativa disponível quando o tamanho das instâncias é moderado ou quando as restrições são suficientemente fortes para permitir poda agressiva.

Finalmente, o presente trabalho justifica-se pela necessidade de sistematizar o conhecimento sobre a técnica em um formato acessível ao estudante universitário de língua portuguesa. Embora haja abundante material em inglês os clássicos de Cormen et al., de Knuth, de Sedgewick a barreira do idioma e, mais importante, o estilo por vezes excessivamente condensado destas obras, tornam necessário um tratamento mais extenso e didático, com exemplos detalhados e linguagem adequada ao público-alvo.

2 CAPÍTULO I: FUNDAMENTAÇÃO TEÓRICA

2.1 Definição Formal e Breve Histórico

O termo "backtracking" foi introduzido pelo matemático Derrick Henry Lehmer na década de 1950. Lehmer, que trabalhava com problemas de teoria dos números envolvendo a busca por soluções inteiras para sistemas de equações, percebeu que poderia acelerar dramaticamente suas computações se, ao invés de gerar todas as combinações possíveis, ele as construísse passo a passo e abandonasse precocemente aquelas que já violassem alguma condição.

A técnica, no entanto, tem raízes mais antigas. O matemático inglês Arthur Cayley, em meados do século XIX, já descrevia métodos sistemáticos para percorrer estruturas combinatórias que hoje reconheceríamos como formas primitivas de backtracking. E, antes de qualquer formalização matemática, xadrezistas e solucionadores de quebra-cabeças já empregavam a estratégia intuitiva: tente um movimento, prossiga, e se o futuro revelar um problema, volte atrás e tente outro.

Formalmente, definimos backtracking como um algoritmo de busca sistemática e exaustiva que constrói soluções incrementalmente e abandona um caminho de construção (realizando um retrocesso) tão logo determina que este caminho não pode levar a uma solução válida completa. A palavra "exaustiva" é crucial: diferentemente de heurísticas que se contentam com soluções aproximadas, o backtracking, quando corretamente implementado, não perde nenhuma solução. Ele pode ser lento, mas é completo.

2.2 A Árvore De Busca Como Modelo De Compreensão

Para compreender o backtracking, precisamos de uma imagem mental apropriada. A imagem mais útil é a da árvore de busca. Cada nó desta árvore representa um estado parcial do problema: uma sequência de decisões já tomadas, do início do processo até aquele ponto. A raiz da árvore (nível zero) representa o estado anterior a qualquer decisão. As folhas representam estados terminais: ou soluções completas (quando todas as decisões foram tomadas e todas as restrições são satisfeitas) ou becos sem saída (quando uma decisão inviabilizou qualquer continuação).

O algoritmo de backtracking percorre esta árvore em profundidade (depth-first). Começa na raiz e desce por um ramo, tomando decisões sucessivas. Ao atingir uma folha, ou ao encontrar um nó que a função de rejeição identifica como impossível de completar, o algoritmo retrocede para o nó ancestral mais recente que ainda possui alternativas não exploradas, e então desce por um ramo diferente.

A figura mental é a de um explorador em um labirinto que, a cada bifurcação, escolhe um caminho e marca os outros para exploração futura. Ao encontrar um beco sem saída, retorna à bifurcação mais recente e tenta o próximo caminho. O explorador só retorna a bifurcações mais antigas quando todos os caminhos das bifurcações mais recentes já foram exauridos.

Esta representação como árvore tem a vantagem de tornar visível o fenômeno da poda. Quando, em determinado nó, a função de rejeição declara que "nenhuma extensão deste nó pode levar a uma solução", todo o sub-ramo abaixo daquele nó é simplesmente ignorado não é gerado, não é percorrido, não consome tempo. O ganho de eficiência é proporcional ao tamanho dos sub-ramos podados.

2.3 Recursão e Backtracking

Um dos pontos que mais geram confusão entre estudantes é a distinção entre recursão e backtracking. Não raro, os dois termos são usados como sinônimos o que é tecnicamente impreciso e pedagogicamente prejudicial.

A recursão é uma propriedade de certos algoritmos e funções: uma função é recursiva se, durante sua execução, ela invoca a si mesma, direta ou indiretamente. A recursão é uma técnica de programação, uma maneira de estruturar a solução de um problema expressando-o em termos de versões menores de si mesmo. Não há, na recursão pura, nenhum elemento de "volta atrás" ou de reconsideração de escolhas anteriores.

O backtracking por outro lado, é uma estratégia algorítmica que quase sempre utiliza recursão como mecanismo de implementação, mas adiciona um ingrediente essencial: a capacidade de desfazer escolhas realizadas em níveis mais profundos da recursão antes de retornar a níveis mais superficiais. É este desfazer este retrocesso que caracteriza a técnica.

Em resumo: toda implementação de backtracking é recursiva (embora se possa simular a recursão com pilha explícita), mas nem todo algoritmo recursivo é de backtracking. O que transforma uma recursão simples em backtracking é a presença da operação de desfazer, que permite explorar múltiplas alternativas no mesmo espaço de estados compartilhado.

2.4 As Funções Constituintes Do Algoritmo

A literatura especializada decompõe o algoritmo de backtracking em quatro componentes funcionais, cujas implementações concretas variam conforme o problema:

2.4.1 A função de rejeição (reject)

É o coração da eficiência do método. Dado um candidato parcial (uma sequência de decisões já tomadas), esta função decide se é possível que alguma extensão deste candidato venha a ser uma solução completa. Se a resposta é "não" (caminho inviável), o algoritmo abandona imediatamente todo o sub-ramo abaixo daquele nó. Quanto mais cedo a inviabilidade for detectada, maior a poda e menor o esforço computacional. A função de rejeição pode ser exata (identifica toda inviabilidade) ou aproximada (identifica apenas um subconjunto da inviabilidade), mas jamais pode cometer o erro oposto declarar viável um caminho que é, de fato, inviável. Isso levaria à perda de soluções, violando a completude do método.

2.4.2 A função de aceitação (accept)

Determina se um candidato é, ele próprio, uma solução completa e válida para o problema. Diferentemente da rejeição, que opera sobre candidatos parciais, a aceitação tipicamente se aplica quando o candidato atingiu o tamanho máximo (todas as decisões foram tomadas) e atende a todas as restrições.

2.4.3 A função de geração da primeira extensão (first)

Produz a primeira alternativa a ser explorada a partir de um dado candidato. A ordem em que as alternativas são geradas tem impacto significativo sobre o desempenho, especialmente quando combinada com heurísticas que buscam primeiro os caminhos mais promissores.

A função de geração da próxima extensão (next)

Produz, a partir de uma extensão já explorada, a extensão seguinte na ordem estabelecida. Quando não há mais extensões a explorar, esta função retorna um valor nulo, sinalizando que o algoritmo deve retroceder ao nível anterior.

O padrão algorítmico que combina estas quatro funções é surpreendentemente simples — e surpreendentemente poderoso. A complexidade está não no algoritmo em si, mas na engenharia das funções de rejeição e de geração para cada domínio específico.

2.5 Onde o Backtracking se aplica

Nem todo problema computacional se presta a uma solução por backtracking. Identificar as condições de aplicabilidade é tão importante quanto dominar a técnica em si.

O backtracking é particularmente adequado para problemas que exibem as seguintes características:

2.5.1 Estrutura de decisão sequencial:

O problema pode ser decomposto em uma sequência de decisões, cada uma tomada a partir de um conjunto finito de alternativas. A ordem pode ser natural (posições de um tabuleiro, células de uma matriz, variáveis de um sistema) ou artificialmente imposta (como ordenar os elementos de um conjunto para gerar subconjuntos).

2.5.2 Testabilidade precoce:

A inviabilidade de um caminho pode ser detectada antes que todas as decisões sejam tomadas. Quanto mais cedo a detecção ocorrer, maior o benefício da poda. Problemas onde as restrições envolvem apenas um subconjunto das variáveis (como restrições binárias em CSPs) são especialmente favoráveis.

2.5.3 Domínios pequenos

O número de alternativas por decisão deve ser gerenciável. Backtracking com fator de ramificação na casa dos milhares torna-se rapidamente impraticável, a menos que as restrições sejam extremamente fortes e a poda quase instantânea.

2.5.4 Soluções completas necessárias:

Quando o problema exige a exibição de todas as soluções, ou a contagem exata do número de soluções, o backtracking oferece a garantia de completude que heurísticas e métodos aproximados não fornecem.

2.6 Onde o backtracking Não Se Aplica

2.6.1 Problemas de otimização contínua

Quando as variáveis são contínuas (números reais) ou os domínios são infinitos, o backtracking, em sua forma padrão, não se aplica. Outros métodos (gradiente descendente, programação não-linear) são necessários.

2.6.2 Problemas onde a poda é ineficaz

Se a função de rejeição nunca ou raramente identifica inviabilidades precoces, o backtracking degenera para força bruta. Neste caso, métodos mais sofisticados (programação dinâmica para certas estruturas, branch-and-bound melhorado, meta-heurísticas) podem ser mais apropriados.

2.6.3 Problemas que admitem algoritmos gulosos ótimos

Se o problema pode ser resolvido por um algoritmo guloso (como a escolha de atividades em uma agenda), o backtracking é desnecessariamente complexo e ineficiente.

2.6.4 Problemas muito grandes

Para instâncias de grande escala de problemas NP-completos, o backtracking, mesmo com poda agressiva, pode simplesmente não terminar em tempo hábil. Nestes casos, aceita-se uma solução aproximada (heurística) ou recorre-se a métodos paralelos/distribuídos.

2.7 Poda

A poda é o mecanismo que distingue o backtracking da enumeração exaustiva. Sem poda, o algoritmo visita todos os nós da árvore de busca, e seu tempo de execução é exatamente proporcional ao tamanho da árvore $\backslash O(b^n) \backslash$ no pior caso. Com poda efetiva, o número de nós visitados pode reduzir-se dramaticamente, tornando tratáveis problemas que, de outra forma, seriam intratáveis.

A poda ocorre quando a função de rejeição, aplicada a um nó, retorna verdadeiro. Neste momento, todo o sub-ramo abaixo daquele nó é ignorado. O ganho é igual ao número de nós no sub-ramo podado multiplicado pelo custo por nó.

A arte do backtracking, em grande medida, é a arte de projetar funções de rejeição que detectem inviabilidade tão cedo quanto possível, com o menor custo computacional possível. Estes dois objetivos detecção precoce e baixo custo frequentemente entram em conflito: uma função de rejeição muito sofisticada pode detectar inviabilidades mais cedo, mas o custo de aplicá-la a cada nó pode anular o ganho obtido com a poda.

2.8 Tipos de Poda

Diferentes estratégias de poda são empregadas conforme a natureza do problema:

2.8.1 Poda por restrições locais

É a forma mais simples. Cada decisão tomada é imediatamente verificada contra as restrições que envolvem apenas as variáveis já atribuídas. Se alguma restrição é violada, o ramo é podado. Esta é a estratégia padrão em problemas de satisfação de restrições (CSPs).

2.8.2 Poda por forward checking

vai um passo além: após atribuir um valor a uma variável, o algoritmo examina as variáveis ainda não atribuídas que possuem restrições com a variável recém-atribuída. Se alguma destas variáveis ficar sem nenhum valor em seu domínio que seja compatível com a atribuição atual, o ramo é podado. O forward checking é mais caro que a verificação local, mas frequentemente produz podas mais precoces.

2.8.3 Poda por limite (branch and bound)

É utilizada em problemas de otimização. O algoritmo mantém o registro da melhor solução encontrada até o momento. Para cada nó, calcula-se um limite superior (ou inferior) do valor que pode ser alcançado a partir daquele nó. Se este limite for pior do que a melhor solução já conhecida, o nó é podado.

2.8.4 Poda por simetria

explora o fato de que, em muitos problemas, diferentes caminhos na árvore de busca levam a estados essencialmente equivalentes. Se o algoritmo detecta que o nó atual é simétrico a um nó já explorado anteriormente (e que já se mostrou infrutífero), o não é podado. Esta técnica é sofisticada, mas pode produzir ganhos dramáticos em problemas altamente simétricos, como o preenchimento de Sudoku.

2.9 Complexidade

Discutir complexidade de algoritmos de backtracking é uma tarefa notoriamente difícil. Diferentemente de algoritmos como ordenação ou busca binária, cujo tempo de execução pode ser expresso por funções simples do tamanho da entrada, o backtracking tem desempenho que varia dramaticamente com a estrutura específica da instância.

No pior caso: quando a função de rejeição nunca identifica nenhuma inviabilidade (ou quando o problema não possui restrições que permitam poda), o algoritmo visita cada nó da árvore de busca completa. Se o problema tem $\lfloor n \rfloor$ decisões e cada decisão pode ser tomada de $\lfloor b \rfloor$ maneiras, o número de nós na árvore (considerando raiz, nós internos e folhas) é da ordem de $\lfloor O(b^n) \rfloor$. Esta é uma complexidade exponencial no pior caso e é o melhor que podemos esperar para problemas NP-completos, a menos que $P = NP$.

No melhor caso: quando as restrições são tão fortes que a primeira escolha em cada nível leva diretamente a uma solução e nenhum retrocesso é necessário, o algoritmo visita apenas $\lfloor O(n \cdot b) \rfloor$ nós (um para cada nível). Esta é uma complexidade linear excelente, mas irrealista para a maioria dos problemas interessantes.

O caso médio: é o que realmente importa na prática, mas é também o mais difícil de analisar. Para muitos problemas, o caso médio é muito melhor que o pior caso, mas ainda exponencial no tamanho da instância. A literatura reporta, por exemplo, que o problema das N-rainhas (um clássico do backtracking) tem desempenho médio que cresce mais lentamente que a exponencial ingênua, mas ainda assim cresce suficientemente rápido para tornar o problema intratável para N acima de algumas dezenas.

A complexidade espacial: é mais favorável. Na implementação recursiva padrão (compartilhando a mesma estrutura de dados mutável entre chamadas), o espaço necessário é $\lfloor O(n) \rfloor$ para a pilha de recursão, mais $\lfloor O(s) \rfloor$ para a estrutura de dados que representa o estado parcial. Esta é uma complexidade linear na profundidade do problema excelente e frequentemente negligenciada nas discussões que se concentram exclusivamente no tempo.

2.10 Variantes E Aprimoramentos

O backtracking básico, tal como descrito, é apenas o ponto de partida. Décadas de pesquisa produziram inúmeras variantes e aprimoramentos:

2.10.1 Backjumping

Modifica o comportamento do retrocesso. No backtracking padrão, quando um beco sem saída é encontrado, o algoritmo retrocede exatamente um nível na árvore de busca. O backjumping, por outro lado, analisa qual decisão, entre aquelas tomadas ao longo do caminho, é a verdadeira causa do fracasso. O algoritmo então retrocede diretamente para o nível daquela decisão, saltando sobre níveis intermediários cujas escolhas não contribuíram para o conflito. O ganho pode ser substancial, especialmente em problemas onde restrições não-locais causam conflitos que só se manifestam tardiamente.

2.10.2 Conflict-Directed Backjumping (CBJ)

É uma evolução do backjumping que mantém, para cada nível, o conjunto de níveis que causaram conflitos. Ao encontrar um beco sem saída, o algoritmo identifica o nível mais recente entre aqueles que participaram de algum conflito e retrocede diretamente para ele, propagando os conjuntos de conflito para níveis superiores.

Heurísticas de ordenamento intervêm na ordem pela qual as variáveis são atribuídas e os valores são experimentados. Embora não modifiquem a complexidade no pior caso, heurísticas como MRV (Minimum Remaining Values atribuir primeiro a variável com o menor número de valores restantes) e LCV (Least Constraining Value experimentar primeiro o valor que menos restringe as variáveis ainda não atribuídas) podem melhorar dramaticamente o desempenho médio.

2.10.3 Forward checking com propagação de restrições

Combina o backtracking com a manutenção da consistência de arco após cada atribuição. A cada novo valor atribuído, o algoritmo executa um algoritmo de consistência (como AC-3) para remover valores inconsistentes dos domínios das variáveis ainda não atribuídas. O custo computacional por nó é maior, mas a poda resultante frequentemente compensa.

3 CAPÍTULO II :ESTUDOS DE CASO

3.1 O Problema Das N-Rainhas

O problema das N-rainhas é, provavelmente, o exemplo mais frequentemente citado em textos sobre backtracking, e por boas razões: é suficientemente simples para ser compreendido rapidamente, suficientemente complexo para exigir uma estratégia não-trivial, e suficientemente elegante para ilustrar todos os aspectos fundamentais da técnica.

O problema consiste em posicionar N rainhas em um tabuleiro de xadrez de dimensões $N \times N$ de modo que nenhuma rainha ataque outra. Em termos de restrições: não pode haver duas rainhas na mesma linha, na mesma coluna ou na mesma diagonal (principal ou secundária). Para $N=8$ (o tabuleiro tradicional), existem 92 soluções distintas (não considerando simetrias); para $N=4$, existem 2.

A abordagem por backtracking para este problema é quase didática em sua clareza. As rainhas são posicionadas uma por linha (ou uma por coluna), garantindo desde o início que não haverá duas na mesma linha. A cada novo posicionamento, verifica-se se a coluna escolhida já está ocupada (conflito vertical) e se a diagonal (tanto principal quanto secundária) já está ocupada.

A poda ocorre assim que uma escolha inviável é detectada: se, ao tentar posicionar a rainha da linha i na coluna j , verifica-se que esta coluna já contém uma rainha em alguma linha anterior, ou que a diagonal está ocupada, aquela coluna é rejeitada e a próxima coluna é tentada. Se todas as N colunas forem tentadas sem sucesso, o algoritmo retrocede: remove a rainha da linha anterior e tenta movê-la para a próxima coluna disponível.

A complexidade do problema das N-rainhas tem sido amplamente estudada. O algoritmo de backtracking ingênuo explora $(N!)$ arranjos (permutações das colunas pelas linhas), o que para $N=8$ dá 40.320 possibilidades trivial para um computador moderno. Para $N=20$, no entanto, $20!$ é cerca de $(2,4 \times 10^{18})$, muito além do que o backtracking ingênuo pode explorar. O ponto crucial é que, para o problema das N-rainhas, a poda é extraordinariamente eficaz: o número de nós realmente visitados pelo algoritmo é muito menor que $(N!)$, tornando o problema tratável para N na casa das dezenas, mas ainda assim intratável para N nas centenas.

3.2 O Problema Do Caixeiro Viajante

O problema do caixeiro viajante (Traveling Salesman Problem TSP) é um dos mais estudados em otimização combinatória. Dadas N cidades e as distâncias entre cada par delas, o objetivo é encontrar o percurso mais curto que visite cada cidade exatamente uma vez e retorne à cidade de origem.

O backtracking para o TSP constrói o percurso incrementalmente: parte-se de uma cidade inicial (digamos, a cidade 1), escolhe-se a próxima cidade a visitar (qualquer uma das $N-1$ restantes), depois a seguinte (entre as $N-2$ restantes), e assim por diante. A cada extensão, calcula-se o comprimento parcial do percurso. Se este comprimento parcial já excede o melhor percurso completo encontrado até o momento, o ramo é podado não há como completá-lo para obter um percurso melhor que o já conhecido.

O desempenho do backtracking para o TSP é pobre para instâncias de tamanho moderado: mesmo com poda, o número de percursos explorados cresce fatorialmente com N . Para $N=20$, o número de percursos (desconsiderando simetrias) é $19! / 2$, aproximadamente (6×10^{16}) . O backtracking resolve TSP para N na casa das 15-20, mas rapidamente se torna inviável para valores maiores. Para estes, métodos mais sofisticados (programação dinâmica com técnica de bitmask, ou branch-and-bound mais agressivo) são necessários.

3.3 O Problema Da Mochila (0/1)

O problema da mochila 0/1 é um clássico de otimização com aplicações em logística, alocação de recursos e finanças. Dados N itens, cada um com um peso e um valor, e uma mochila com capacidade máxima C , deseja-se selecionar um subconjunto de itens que maximize o valor total sem exceder a capacidade.

O backtracking para este problema percorre os itens em alguma ordem, decidindo, para cada item, se ele é incluído ou não na mochila. A cada inclusão, verifica-se se o peso total não excede a capacidade. A poda é realizada calculando-se um limite superior otimista para o valor que pode ser alcançado a partir do ponto atual (tipicamente, o valor acumulado mais o valor dos itens restantes, ou uma relaxação fracionária da mochila). Se este limite é inferior ao melhor valor já encontrado.

4 Conclusão

O presente trabalho teve como propósito analisar o backtracking enquanto técnica algorítmica para resolução de problemas computacionais, investigando seus fundamentos, condições de aplicabilidade, variações e limitações. Ao longo da pesquisa, ficou evidente que o backtracking não se reduz a um algoritmo específico, mas constitui uma família de estratégias unificadas por um princípio central: a construção incremental de soluções aliada à capacidade de retroceder quando uma escolha se revela inadequada. Esta simplicidade conceitual, contudo, esconde desafios significativos relacionados ao projeto da função de rejeição e ao equilíbrio entre custo de poda e ganho em eficiência.

Um dos pontos de maior relevância esclarecidos foi a distinção entre recursão e backtracking, frequentemente confundida na literatura introdutória. Demonstrou-se que a recursão é o mecanismo de implementação, enquanto o backtracking é a estratégia que adiciona a operação deliberada de desfazer escolhas anteriores, permitindo explorar múltiplas alternativas sobre uma mesma estrutura de dados compartilhada. Esta compreensão é fundamental para implementações corretas e eficientes.

A análise da complexidade revelou a natureza dual do método. No pior caso, quando as restrições são fracas ou inexistentes, o algoritmo degenera para a enumeração exaustiva, com complexidade exponencial $O(b^n)$ manifestação do fenômeno da explosão combinatória. No melhor caso, a poda eficaz pode reduzir o esforço para próximo do linear. Esta variabilidade não é uma falha, mas uma consequência inevitável da generalidade do método, exigindo do profissional a capacidade de avaliar, para cada problema específico, se o backtracking é a ferramenta adequada.

5 Bibliografia

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2012). *Algoritmos: Teoria e prática* (3ª ed.). Rio de Janeiro: Elsevier.

Goodrich, M. T., & Tamassia, R. (2013). *Estruturas de dados e algoritmos em Java* (5ª ed.). Porto Alegre: Bookman.

Knuth, D. E. (2011). *The art of computer programming* (Vol. 4A). Boston, MA: Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Boston, MA: Addison-Wesley.

Weiss, M. A. (2014). *Data structures and algorithm analysis in Java* (3rd ed.). Upper Saddle River, NJ: Pearson.

Horowitz, E., Sahni, S., & Rajasekaran, S. (2008). *Fundamentals of computer algorithms* (2nd ed.). Hyderabad: Universities Press.

Lafore, R. (2002). *Data structures and algorithms in Java* (2nd ed.). Indianapolis, IN: Sams Publishing.